



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Робототехника и комплексная автоматизация

КАФЕДРА Системы автоматизированного проектирования (РК-6)

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Студент _____ Фёдоров Артемий Владиславович _____
фамилия, имя, отчество

Группа РК6-41М

Тип практики Преддипломная

Название предприятия НУК РК МГТУ им. Н.Э. Баумана

Студент _____ Фёдоров А.В. _____
подпись, дата фамилия, и.о.

Руководитель практики
от кафедры _____ Витюков Ф.А. _____
подпись, дата фамилия, и.о.

Оценка _____

«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой РК6

_____ А.П. Карпенко _____
« ____ » _____ 2026 г.

З А Д А Н И Е
на прохождение производственной практики
Эксплуатационная / Преддипломная
Тип практики

Студент

Фёдоров Артемий Владиславович
Фамилия Имя Отчество

2 курса
№ курса

группы

РК6-41М
индекс группы

в период с 09.02.2026 г. по 21.05.2026 г.

Предприятие: _____ НУК РК МГТУ им. Н.Э. Баумана _____
Подразделение: _____

(отдел/сектор/цех)

Руководитель практики от кафедры:

Витюков Федор Андреевич, старший преподаватель
(Фамилия Имя Отчество полностью, должность)

Задание:

1. Провести анализ UI игровых проектов жанра Top-Down RPG
2. Реализовать систему управления игровым персонажем

Дата выдачи задания 10 февраля 2026 г.

Руководитель практики от кафедры

_____ / Ф.А. Витюков /

Студент

_____ / А.В. Фёдоров /

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1. КРАТКИЙ ОТЧЕТ О ВЫПОЛНЕННЫХ РАБОТАХ.....	6
1.1. Анализ существующих проектов.....	6
1.2. Программная реализация.....	7
2. АНАЛИЗ РЕЗУЛЬТАТОВ.....	7
ЗАКЛЮЧЕНИЕ	8
СПИСОК ЛИТЕРАТУРЫ.....	8

ВВЕДЕНИЕ

Создание интуитивно понятной и эффективной системы управления является одной из ключевых задач при разработке компьютерных игр, в частности в жанре «Ролевая игра с видом сверху» (с англ. Top-Down RPG). От того, насколько точно и удобно игрок может взаимодействовать с игровым миром - выбирать персонажей, активировать объекты, выполнять действия - напрямую зависит пользовательский опыт. В контексте разработки на движке Unreal Engine 5, обладающем глубоким, но сложным инструментарием для обработки ввода, проблема проектирования логики взаимодействия становится особенно актуальной.

Объектом исследования данной работы являются системы управления в компьютерных играх с видом сверху, а именно - реализация взаимодействия с игровым миром посредством кнопок мыши. Предметом исследования выступает возможность реализации полного цикла взаимодействия (выбор персонажа, выбор интерактивного объекта, выбор действия) с использованием исключительно левой кнопки мыши, без привлечения дополнительных клавиш.

Техническая база для реализации систем ввода в Unreal Engine 5 хорошо документирована и даёт разработчику возможность гибко настраивать привязку клавиш и мыши к игровым действиям, а также динамически менять контексты ввода в зависимости от игровой ситуации. Однако документация и примеры фокусируются на технической стороне привязки (как назначить действие на клавишу), а не на проектировании логики взаимодействия, определяющей, какое именно действие должно произойти при клике по тому или иному объекту на сцене.

Для проектирования логики взаимодействия пользователя с игровым миром предлагается провести анализ успешных проектов разных эпох - Fallout (1997), Baldur's Gate (1998), XCOM 2 (2016) и Baldur's Gate 3 (2023) - чтобы выявить ключевые паттерны управления и их применимость для современной разработки.

Целью работы является разработка системы взаимодействия с игровым миром для Top-Down RPG в Unreal Engine 5, позволяющей выполнять операции выбора персонажа, выбора интерактивного предмета и выбора одного из доступных действий с использованием только левой кнопки мыши на основе анализа эволюции управленческих решений в успешных игровых проектах.

1. КРАТКИЙ ОТЧЕТ О ВЫПОЛНЕННЫХ РАБОТАХ

1.1. АНАЛИЗ СУЩЕСТВУЮЩИХ ПРОЕКТОВ

В рамках исследования были проанализированы четыре коммерчески успешных проекта, представляющих различные эпохи развития жанра Top-Down RPG: *Fallout* (1997), *Baldur's Gate* (1998), *XCOM 2* (2016) и *Baldur's Gate 3* (2023). Целью анализа являлось изучение эволюции систем управления мышью, а именно - распределения функций между левой и правой кнопками мыши, а также возможности выполнения всех действий через одну кнопку.

Fallout 1 (1997) использует схему управления, построенную на трёх состояниях курсора, которые переключаются правой кнопкой мыши. Левая кнопка мыши (далее ЛКМ) выполняет действие, соответствующее текущему режиму курсора:

- В режиме перемещения ЛКМ выполняет перемещение персонажа в указанную точку;
- В режиме атаки ЛКМ выполняет атаку игрового персонажа по выбранной цели.
- В режиме взаимодействия ЛКМ выполняет различные действия взаимодействия с игровым миром: разговор с неигровым персонажем, открытие дверей, использование предметов.

Особый интерес представляет последний режим. При удержании левой кнопки мыши в этом режиме открывается круговое меню с дополнительными иконками действий: «говорить», «осмотреть», «использовать», «взять» и другими. Это позволяет игроку выбирать специфическое действие с неигровым персонажем или объектом, при том что обычный клик левой кнопкой мыши в режиме взаимодействия запускает стандартное действие (например, разговор).

В **Baldur's Gate (1998)** прослеживается чёткое разделение функций между левой и правой кнопками мыши, которое впоследствии станет стандартом популярным среди большого количества RPG проектов. Игрок наводит курсор на объект и интерфейс динамически отображает, какое действие будет выполнено при клике ЛКМ посредством изменения иконки указателя мыши. Клик ПКМ, в свою очередь, открывает детальное контекстное меню с полным списком доступных действий для выбранного объекта или персонажа.

Так, в **XCOM 2 (2016)** левая кнопка мыши аналогично используется для выбора персонажа, выбора цели и подтверждения действий. Правая кнопка мыши выполняет функцию отмены выбора и открытия дополнительной информации. В данном проекте также присутствует оригинальная механика, использующая обе кнопки мыши: при выборе определенного персонажа - солдата ближнего боя - игроку даётся возможность совместить перемещение с атакой за счёт одновременного нажатия обоих кнопок мыши по цели.

Baldur's Gate 3 (2023), как самый современный представитель жанра Top-Down RPG представляет собой самый релевантный объект для анализа, так как он разрабатывался с учётом современных стандартов UI и UX. В данном проекте используется уже устоявшийся принцип: нажатие левой кнопки мыши для основного действия, зависящего от контекста (перемещение, разговор с персонажем, взаимодействие с объектом) и нажатие правой кнопки мыши для вызова контекстного меню со списком всех доступных действий.

Проведённый анализ четырёх коммерчески успешных проектов разных эпох позволяет выявить чёткую эволюционную линию в развитии систем управления Top-Down RPG. Ранние проекты требовали ручного переключения режимов курсора, что создавало дополнительную когнитивную нагрузку на игрока. На смену этому подходу пришла модель контекстно-зависимого взаимодействия, впоследствии ставшая стандартом.

На основе анализа принято решение реализовать в разрабатываемой системе взаимодействия модель, аналогичную Baldur's Gate 3: левая кнопка мыши будет выполнять контекстно-зависимое основное действие (выбор персонажа, выбор интерактивного предмета, использование объекта по умолчанию), а вызов меню со списком всех доступных действий будет осуществляться через дополнительный модификатор (удержание ЛКМ или нажатие ПКМ).

1.2. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Для корректной обработки ввода в зависимости от текущего игрового контекста была разработана система состояний пользователя, реализованная в классе *TopDownRPGCharacterController*, наследующим от *APlayerController*. *APlayerController* - стандартный класс, нужный для взаимодействия игрока с игровым миром. Разработанная система описывает следующие состояния:

- None - ничего не выбрано (нейтральное состояние)
- UnitSelected - игровой персонаж выбран (ожидание команды)
- ActionTargeting - выбрана способность персонажа (ожидание выбора цели для атаки)
- Busy - выбранный персонаж выполняет действие (ожидание завершения действия)

На рис.1 представлена схема переходов между различными состояниями игрока.

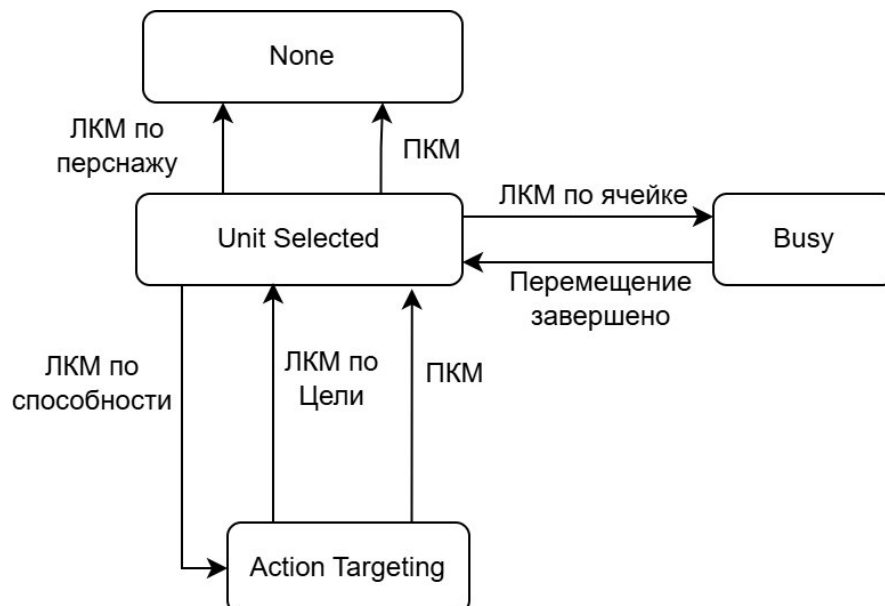


Рисунок 1 – Система состояний игрока

В классе `ATopDownRPGCharacterController` реализованы функции `handleStateChange`, `handleLeftClick`, `handleRightClick`, поведение которых зависит от текущего состояния игрока. При переходе между состояниями система выполняет ряд вспомогательных действий, обеспечивающих визуальную обратную связь и подготовку интерфейса к следующему шагу взаимодействия:

- При входе в состояние `UnitSelected`: `Character Controller` получает список доступных способностей выбранного юнита и отображает их в пользовательском интерфейсе; также на выбранного персонажа помещается визуальный маркер (светящийся круг), сигнализирующий игроку, кто именно выбран в данный момент.
- При входе в состояние `ActionTargeting`: `Character Controller` анализирует выбранную способность и подсвечивает те клетки игрового поля, которые могут быть целями для данной способности (например, все враги в радиусе действия или все союзники).
- При выходе из любого состояния: `Character Controller` очищает временные визуальные эффекты - убирает подсветку клеток, скрывает маркер с персонажа (или передаёт его новому выбранному юниту) и обновляет интерфейс, убирая панель способностей, если она больше не актуальна.

1.1 Разработка виджетов

Для отображения информации о персонаже и предоставления доступа к его способностям были разработаны два пользовательских виджета, реализованных на базе класса `UUserWidget` - стандартного средства создания интерфейса в Unreal Engine. Первый виджет - `UCharacterInfoWidget` - выполняет роль контейнера, отображающего общую информацию о выбранном персонаже: его имя, текущее здоровье, портрет и другие ключевые характеристики. Внутри данного виджета располагается элемент `TileView`, предназначенный для динамического отображения списка доступных способностей персонажа. Каркас виджета `UCharacterInfoWidget` представлен на рис.2.

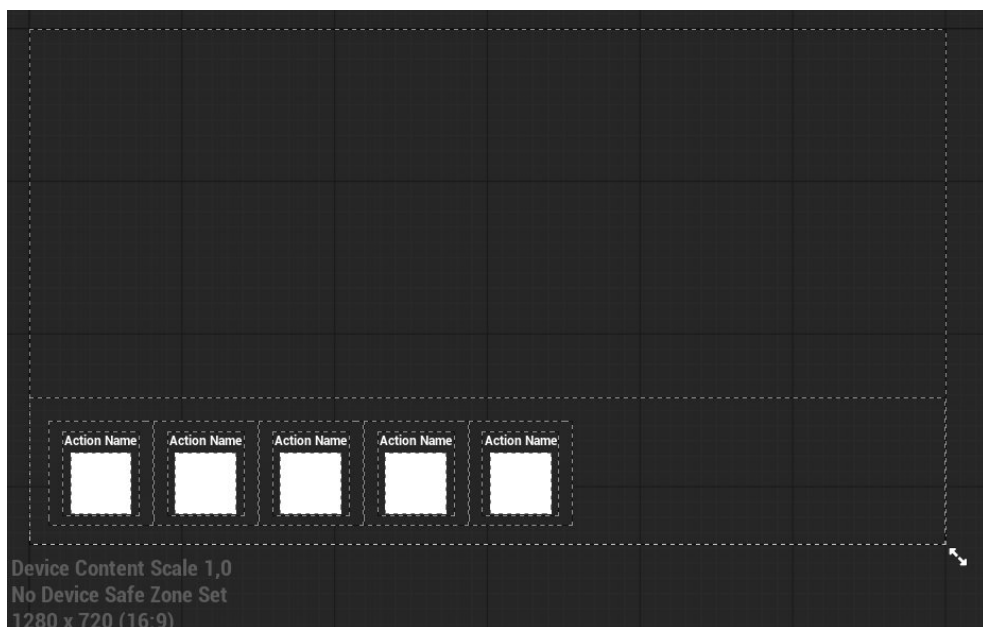


Рисунок 2 – Виджет `UCharacterInfoWidget` в редакторе UMG

Второй виджет - `UCharacterActionWidget` - представляет собой элемент списка, отображающий отдельную способность персонажа. Виджет реализует интерфейс `IUserObjectListEntry`, что позволяет ему корректно работать в составе `TileView` и автоматически заполняться данными из привязанного к нему объекта (название способности, иконка, дальность). Структура виджета `UCharacterActionWidget` представлена на рис.3. Полученный HUD представлен на рис. 4.

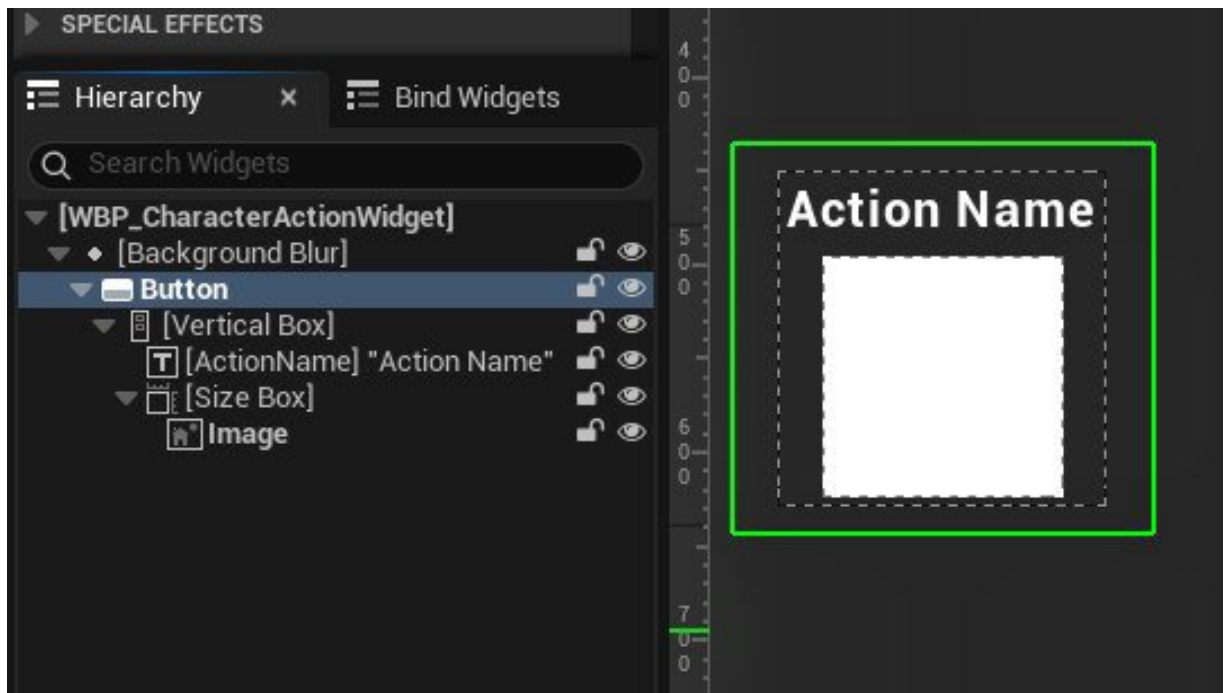


Рисунок 3 – Виджет *UCharacterActionWidget* в редакторе *UMG*

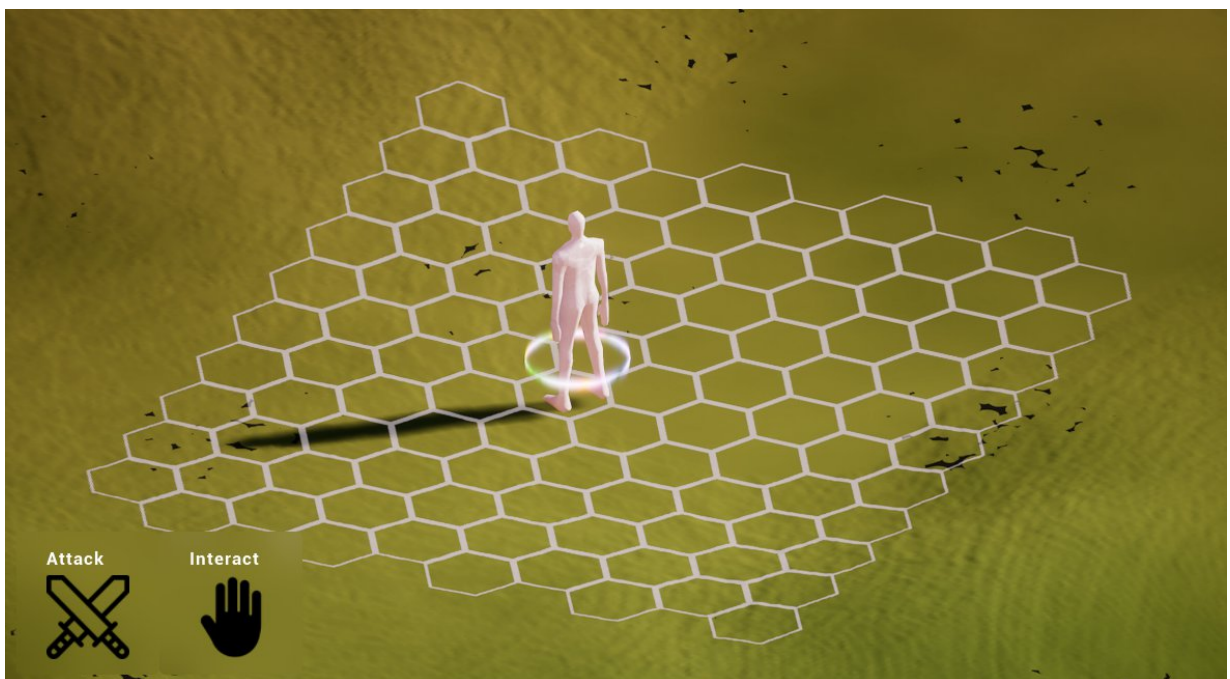


Рисунок 4 – Виджеты, отображаемые в HUD при запуске игры.

1.2 Связывание пользовательского интерфейса с разработанными игровыми механиками

Процесс заполнения интерфейса происходит следующим образом. При клике игрока по персонажу срабатывает логика выбора в контроллере (*TopDownRPGCharacterController*), и система переходит в состояние *UnitSelected*. В этот момент контроллер обращается к компоненту способностей выбранного персонажа (*UCharacterAbilitySystem*) и получает массив доступных для него способностей (*UCharacterAbility*). Полученный массив передаётся в *TileView*, который автоматически создаёт для каждого элемента массива экземпляр виджета *UCharacterActionWidget* (используя его как шаблон элемента). Каждый созданный виджет получает ссылку на соответствующий объект способности и отображает его визуальное представление - иконку, название, дальность и другие параметры.

Делегаты в Unreal Engine представляют собой механизм вызова методов, позволяющий реализовать паттерн «observer» (Наблюдатель). Делегат объявляет, что некоторое событие произошло, а все подписанные на это событие объекты получают уведомление. Unreal Engine предоставляет несколько видов делегатов: однозначные (один подписчик), многозначные (множество подписчиков) и динамические (поддерживающие сериализацию и работу в *Blueprints*). В данной работе используются многозначные делегаты, позволяющие при необходимости расширять количество подписчиков без изменения кода.

При создании каждого экземпляра виджета *UCharacterActionWidget* создаётся делегат, инкапсулирующий действие, связанное с данной способностью. Контроллер игрока (*TopDownRPGCharacterController*) подписывается на этот делегат в момент создания виджета. Когда игрок нажимает на кнопку способности в интерфейсе, делегат срабатывает и передаёт в контроллер объект класса *UCharacterAbility*, соответствующий выбранной способности. Контроллер, получив этот объект, переходит в состояние *ActionTargeting*, запоминая выбранную способность для последующего применения к указанной игроком цели.

С помощью инструментов отладки сетки, разработанных ранее, реализована возможность отображения “зоны поражения” выбранной способности. Используя эвристическую функцию оценки расстояния между клетками и “дальность” конкретной способности, класс сетки подсвечивает клетки, на которые может быть применена способность. Демонстрация представлена на рис. 5.

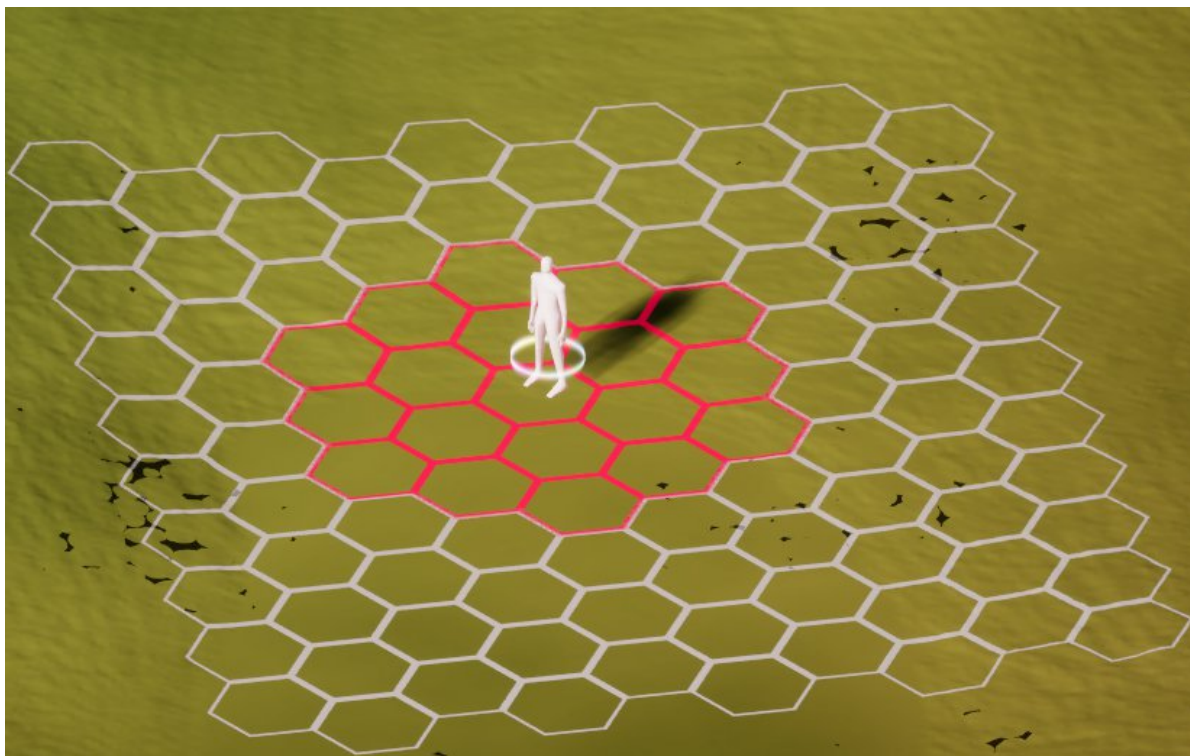


Рисунок 5 – Отображение “зоны поражения” способности.

2. АНАЛИЗ РЕЗУЛЬТАТОВ

В ходе работы разработана и протестирована система контекстно-зависимого управления левой кнопкой мыши. Анализ результатов проводился по трём направлениям.

1. Реализованная система полностью повторяет модель Baldur's Gate 3: левая кнопка мыши выполняет основное действие в зависимости от контекста (выбор персонажа, использование объекта, перемещение). Дополнительные действия доступны через удержание левой кнопки или правую кнопку мыши. Требование «одна кнопка для основных действий» выполнено.
2. Состояния None, UnitSelected, ActionTargeting, Busy переключаются правильно во всех тестовых сценариях (выбор персонажа, выбор способности, выбор цели, отмена, выполнение). При входе в ActionTargeting система подсвечивает допустимые клетки (например, всех врагов в радиусе 2 клеток). При выходе из состояний временная подсветка и маркеры очищаются.
3. Виджеты UCharacterInfoWidget и UCharacterActionWidget динамически отображают информацию о персонаже и список его способностей через TileView. Делегаты связывают клик по кнопке с контроллером, передавая объект способности. Подсветка зоны поражения даёт игроку понятную визуальную подсказку.

ЗАКЛЮЧЕНИЕ

В ходе практики выполнен анализ эволюции систем управления в проектах Fallout, Baldur's Gate, XCOM 2 и Baldur's Gate 3, на основе которого реализована контекстно-зависимая модель взаимодействия левой кнопкой мыши в Unreal Engine 5. Разработан конечный автомат состояний игрока, динамический интерфейс отображения способностей и механизм подсветки целей. Поставленная цель достигнута: система позволяет выполнять выбор персонажа, выбор объектов и активацию действий одной левой кнопкой мыши. Результаты могут использоваться как основа для управления в проектах Top-Down RPG.

СПИСОК ЛИТЕРАТУРЫ

1. Unreal Engine 4 Documentation // Unreal Engine Documentation
URL: <https://docs.unrealengine.com/>. Дата обращения: 15.02.2026.
2. RPG Maker // Kadokawa Games URL: <https://www.rpgmakerweb.com/>. Дата обращения: 17.02.2026.
3. Turn Based Strategy Framework // Crooked Head
URL: <https://assetstore.unity.com/packages/templates/systems/turn-based-strategy-framework-50282>. Дата обращения: 18.02.2026.
4. Unity Tilemap System // Unity Documentation URL: <https://docs.unity.com/en-us>. Дата обращения: 19.02.2026.
5. Advanced Turn Based Tile Toolkit Plugin // Knut Øverbye
URL: <https://unrealmonster.com/advanced-turn-based-tile-toolkit-5-0-5-3/>. Дата обращения: 20.02.2026.
6. Gameplay Ability System for Unreal Engine // Epic Developer Community.
URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system-for-unreal-engine>. Дата обращения: 10.03.2026.